

ROUND ROBIN VS SJF SCHEDULING METHODS

DONE BY:

CHECKED BY:

INTRODUCTION: BRIEF DESCRIPTION OF THE ALGORITHMS:

Round Robin is a scheduling method used by operating systems to manage the execution time of multiple processes that are competing for CPU usage. The system rotates through all processes, allocating each of them a fixed time slice, called **quantum time**, regardless of their priority.

The primary goal of this scheduling algorithm is to certify that all processes are given an equal opportunity to execute, which should provide fairness among tasks.

SJF stands for **Shortest Job First**. It is a scheduling method that selects the waiting process with the smallest execution time to execute first. In the **preemptive** version of this algorithm, the processor is allocated to the job closest to completion.

BRIEF SUMMARY:

This documentation's purpose is to provide evidence on the **comparison** between the **Round Robin** and the **Shortest Job First (SJF)** scheduling methods used by operating systems. It provides executed examples with picture evidence taken from its website counterpart. This document uses natural language, diagrams, and tabular structures to clarify and explain the context.

It will provide comparison between the quality and fairness of each of the mentioned scheduling methods.

ROUND ROBIN

SCENARIO 1: NORMAL CASE (ROUND ROBIN)

Number of Processes: 4

Quantum Time (Q): 3

Processes	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

STEP BY STEP EXECUTION:

From 0 to 3:

1. Process 1 enters queue and starts executing for 3 units.
2. Process 2 enters ready queue behind Process 1 at AT=1.
3. Process 3 enters ready queue behind Process 2 at AT=2.
4. Process 4 enters ready queue behind Process 3 at AT=3.
5. System detects that Process 1 has exhausted its 3 units and pauses it.
6. Process 1 is put behind Process 4 in the ready queue.
7. Remaining burst time for Process 1: $8-3=5$.

From 3 to 6:

1. Process 2 starts executing for 3 units.
2. System detects that Process 2 has exhausted its 3 units and pauses it.
3. Process 2 is put behind Process 1 in the ready queue.
4. Remaining burst time for Process 2: $4-3=1$.

From 6 to 9:

1. Process 3 starts executing for 3 units.
2. System detects that Process 3 has exhausted its 3 units and pauses it.
3. Process 3 is put behind Process 2 in the ready queue.
4. Remaining burst time for Process 3: $9-3=6$.

From 9 to 12:

1. Process 4 starts executing for 3 units.
2. System detects that Process 4 has exhausted its 3 units and pauses it.
3. Process 4 is put behind Process 3 in the ready queue.
4. Remaining burst time for Process 4: $5-3=2$.

From 12 to 15:

1. Process 1 resumes execution for 3 units.
2. System detects that Process 1 has exhausted its 3 units and pauses it.
3. Process 1 is put behind Process 4 in the ready queue.
4. Remaining burst time for Process 1: $5-3=2$.

From 15 to 16:

1. Process 2 resumes execution for 3 units.
2. System detects that Process 2 has ended at 16.
3. Ready Queue becomes [P3, P4, P1]

From 16 to 19:

1. Process 3 resumes execution for 3 units.
2. System detects that Process 3 has exhausted its 3 units and pauses it.
3. Process 3 is put behind Process 1 in the ready queue.
4. Remaining burst time for Process 3: $6-3=3$.

From 19 to 21:

1. Process 4 resumes execution for 3 units.
2. System detects that Process 4 has ended at 21.
3. Ready Queue becomes [P1, P3].

From 21 to 23:

1. Process 1 resumes execution for 3 units.
2. System detects that Process 1 has ended at 23.
3. Ready Queue becomes [P3]

From 23 to 26:

1. Process 3 resumes execution for 3 units.
2. System detects that Process 3 has ended at 26.
3. CPU is released.

AVERAGE TURN AROUND TIME AND AVERAGE WAITING TIME:

Processes	Turn Around Time	Waiting Time
P1	23	15
P2	15	11
P3	24	15
P4	18	13
Averages	20	13.5

GANTT CHART:

SCENARIO 2: FAIRNESS CASE (ROUND ROBIN)

Number of Processes: 5

Quantum Time (Q): 4

Processes	Arrival Time	Burst Time
P1	0	20
P2	1	3
P3	2	3
P4	3	3
P5	4	2

STEP BY STEP EXECUTION:

From 0 to 4:

1. Process 1 enters and starts executing for 4 units.
2. Process 2 enters the ready queue behind Process 1 at AT=1.
3. Process 3 enters the ready queue behind Process 2 at AT=2
4. Process 4 enters the ready queue behind Process 3 at AT=3.
5. Process 5 enters the ready queue behind Process 4 at AT=4.
6. System detects that Process 1 has exhausted its 4 units and pauses it.
7. Process 1 is put behind Process 5 in the ready queue.
8. Remaining burst time for Process 1: $20-4=16$.

From 4 to 7:

1. Process 2 starts execution for 4 units.
2. System detects that Process 2 has ended at 7.
3. Ready Queue becomes: [P3, P4, P5, P1].

From 7 to 10:

1. Process 3 starts execution for 4 units.
2. System detects that Process 3 has ended at 10.
3. Ready Queue becomes: [P4, P5, P1]

From 10 to 13:

1. Process 4 starts execution for 4 units.
2. System detects that Process 4 has ended at 13.
3. Ready Queue becomes: [P5, P1].

From 13 to 15:

1. Process 5 starts execution for 4 units.
2. System detects that Process 5 has ended at 15.
3. Ready Queue becomes: [P1].

From 15 to 19:

1. Process 1 resumes execution for 4 units.
2. System detects that Process 1 has exhausted its 4 units and pauses it.
3. Process 1 returns to the ready queue.
4. Remaining burst time of Process 1: $16-4=12$.

From 19 to 23:

1. Process 1 resumes execution for 4 units.
2. System detects that Process 1 has exhausted its 4 units and pauses it.
3. Process 1 returns to the ready queue.
4. Remaining burst time of Process 1: $12-4=8$.

From 23 to 27:

1. Process 1 resumes execution for 4 units.
2. System detects that Process 1 has exhausted its 4 units and pauses it.
3. Process 1 returns to the ready queue.
4. Remaining burst time of Process 1: $8-4=4$.

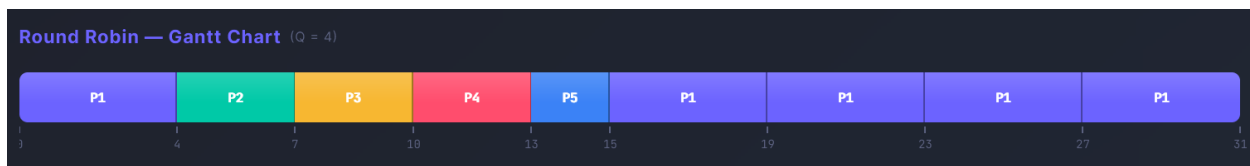
From 27 to 31:

1. Process 1 resumes execution for 4 units.
2. System detects that Process one has ended at 31.
3. CPU is released.

AVERAGE TURN AROUND TIME AND AVERAGE WAITING TIME:

Processes	Turn Around Time	Waiting Time
P1	31	11
P2	6	3
P3	8	5
P4	10	7
P5	11	9
Averages	13.2	7

GANTT CHART:



SCENARIO 3: INVALID INPUT CASE (ROUND ROBIN)

The system detects invalid instances and returns error warnings. Negative quantum time, invalid time metrics, and repetition of processes are all detected by the system. It will now run any simulation unless the user provides correct input.

Processes Included: 3

Quantum Time: -1

Process	Arrival Time	Burst Time
P1	0	5
P2	-2	3
P1	1	0

STEP BY STEP EXECUTION:

1. System starts running.
2. System detects invalid Time Quantum -1, invalid negative input for Process 2's arrival time, and invalid zero input for Process 1's burst time and its duplication.
3. System returns error warning and aborts.

AVERAGE TURN AROUND TIME AND AVERAGE WAITING TIME:

Processes	Turn Around Time	Waiting Time
Averages	Not applicable	Not applicable

GANTT CHART:

No gantt chart generated due to invalid inputs.

RESULT OF ATTEMPT:

⚠ Please fix the following errors:

- Row 2: Duplicate Process ID "P1".
- Row 2 (P1): Arrival Time cannot be negative.
- Row 3 (P3): Burst Time must be > 0.
- Quantum must be > 0.

SHORTEST JOB FIRST (PREEMPTIVE)

SCENARIO 1: NORMAL CASE (SJF)

Processes Included:

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

STEP BY STEP EXECUTION:

From 0 to 1:

1. Process 1 enters and executes for 1 unit.
2. System detects Process 2 has entered.
3. System compares Process 1 and Process for the shortest duration.
4. System preempts Process 1.
5. Remaining burst time for Process 1: $8-1=7$.

From 1 to 5:

1. Process 2 starts executing for 4 units.
2. At 2, Process 3 arrives.
3. System compares Process 3 with Process 2 and preempts Process 3.
4. At 3, Process 4 arrives.
5. System compares Process 4 with Process 2 and preempts Process 4.
6. System detects that Process 2 has ended at 5.

From 5 to 10:

1. System analyses ready queue [P3, P4, P1] and detects that the shortest duration is Process 4.
2. Process 4 starts executing for 5 units at 5.
3. System detects that Process 4 has ended at 10.

From 10 to 17:

1. System analyses ready queue [P3, P1] and detects that the shortest duration is Process 1.
2. Process 1 starts executing for 7 units at 10.
3. System detects that Process 1 has ended at 17.

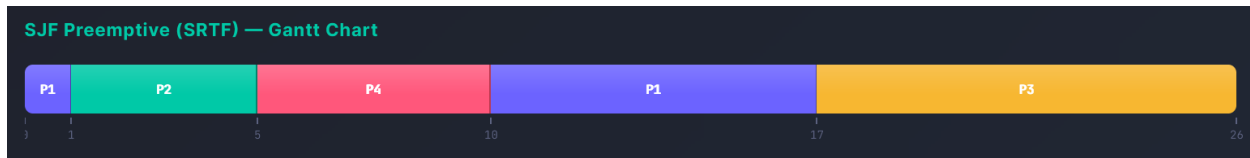
From 17 to 26:

1. System only detects Process 3 in the ready queue.
2. Process 3 starts executing for 9 units at 17.
3. System detects that Process 3 has ended at 26.
4. CPU is released.

AVERAGE TURN AROUND TIME AND AVERAGE WAITING TIME:

Processes	Turn Around Time	Waiting Time
P1	17	9
P2	4	0
P3	24	15
P4	7	2
Averages	13	6.5

GANTT CHART:



SCENARIO 2: FAIRNESS CASE (SJF)

Processes Included: 5

Process	Arrival Time	Burst Time
P1	0	20
P2	1	3
P3	2	3
P4	3	3
P5	4	2

STEP BY STEP EXECUTION:

From 0 to 1:

1. Process 1 enters and starts executing for 1 unit.
2. System detects that Process 2 has entered the ready queue and compares between it and Process 1.
3. System detects that Process 2 has a shorter duration than Process 1 and preempts process 1.
4. Remaining burst time for Process 1: $20-1=19$.

From 1 to 4:

1. Process 2 starts executing for 3 units.
2. At 2, Process 3 has entered the ready queue.
3. System compares it with Process 2 and finds the two equivalent.
4. At 3, Process 4 has entered the ready queue.
5. System compares it with Process 2 and finds the two equivalent.
6. At 4, Process 2 has finished execution.

From 4 to 6:

1. Process 5 has entered the ready queue [P1, P3, P4, P5].
2. System detects that Process 5 is the shortest and it executes for 2 units at 4.
3. System detects that Process 5 has ended at 6.

From 6 to 9:

1. System analyses the ready queue [P1, P3, P4] and executes Process 3 due to earlier arrival time.
2. Process 3 executes for 3 units at 6.
3. System detects that Process 3 has ended at 9.

From 9 to 12:

1. System analyses the ready queue [P1, P4] and detects that Process 4 has the shortest duration.
2. Process 4 executes for 3 units at 9.
3. System detects that Process 4 has ended at 12.

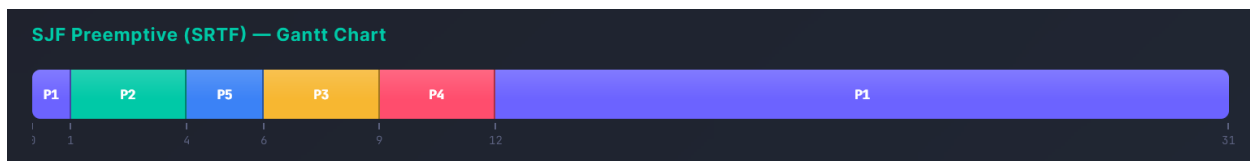
From 12 to 31:

1. System detects that only Process 1 is available in the ready queue.
2. Process 1 resumes execution at 12.
3. System detects that Process 1 has ended execution at 31.
4. CPU is released.

AVERAGE TURN AROUND TIME AND AVERAGE WAITING TIME:

Processes	Turn Around Time	Waiting Time
P1	31	11
P2	3	0
P3	7	4
P4	9	6
P5	2	0
Averages	10.4	4.2

GANTT CHART:



SCENARIO 3: INVALID INPUT CASE (SJF):

The system detects invalid entries and returns error warnings. It will not run the simulation unless correct inputs are given. Invalid time metrics trigger the error warning.

Processes Included: 3

Process	Arrival Time	Burst Time
P1	0	5
P2	-2	3
P1	1	0

STEP BY STEP EXECUTION:

1. System starts running.
2. System detects invalid Time Quantum -1, invalid negative input for Process 2's arrival time, and invalid zero input for Process 1's burst time and its duplication.
3. System returns error warning and aborts.

AVERAGE TURN AROUND TIME AND AVERAGE WAITING TIME:

Processes	Turn Around Time	Waiting Time
Averages	Not applicable	Not applicable

GANTT CHART:

No gantt chart generated due to invalid input.

RESULT OF ATTEMPT:

⚠ Please fix the following errors:

- Row 2: Duplicate Process ID "P1".
- Row 2 (P1): Arrival Time cannot be negative.
- Row 3 (P3): Burst Time must be > 0.
- Quantum must be > 0.

COMPARISON: QUALITY OF THE TWO METHODS:

Quality in CPU is evaluated based on **time metrics** such as **average waiting time, turn-around time, and response time**.

In both Round Robin and SJF scheduling, the **total completion time for all processes is 26 time units**. This is **insufficient** since it is expected that both algorithms execute the same set of processes with no idle CPU time. In conclusion, total time is not a valid measure of quality nor efficiency.

To compare the qualities of Round Robin and SJF methods, we need to compare the average turn-around times, the average waiting times, and the average response time of each of the scheduling methods.

Using the Normal Scenarios for these hypotheses:

Round Robin:

PID	Waiting Time	Turn Around Time	Response Time
P1	15	23	0
P2	11	15	2
P3	15	24	4
P4	13	18	6
Averages	13.5	20	3

SJF (Pre-emptive):

PID	Waiting Time	Turn Around Time	Response Time
P1	9	17	0
P2	0	4	0
P3	15	24	15
P4	2	7	2
Averages	6.5	13	4.25

In the tables above, **significant differences** in the averages appear when the time metrics are calculated. The **SJF** method demonstrates **higher efficiency** by **minimizing the average waiting time and turn-around time**. On the contrary, the **Round Robin** method shows **higher time consumption**.

Shortest Time First (SJF) method **prioritises** executing the **shorter methods** first to cut down waiting times and turn-around times. This makes it **more efficient** than **Round Robin** when it comes to quality of time. Round Robin values **fairness** rather than efficiency by granting each process a specific quota that once it exhausts, the system puts it back into a ready queue and executes the next process for the same quota.

COMPARISON: FAIRNESS OF THE TWO METHODS:

When it comes to terms of **fairness**, **Round Robin** excels at it. The concept of fairness circulates an algorithm's ability to provide equal time for each process. Unlike SJF, it **does not prioritise efficiency** in time or resources.

Round Robin functions in a way in which a certain quota, previously mentioned as **Quantum Time**, is assigned to each process. These processes, all settled within a ready queue, rotate around in a circular concept following the rules of a queue data structure. **It does not allow any process to override another.**

To visually compare between the methods, refer to the Fairness scenarios provided previously.

Some disadvantages of SJF are that there is a chance of **starvation**. A process, even though labelled short, could take a very long time to execute, causing other processes to not execute due to their larger burst time. This issue is **avoided** in Round Robin.

CONCLUSION:

Round Robin is an algorithm that operating systems use when it prioritises fairness among processes rather than time and resource consumption. It gives process an equal time quota through quantum time and therefore achieves its purpose.

The pre-emptive type of SJF, called Shortest Remaining Time First (SRTF), prioritises efficiency and resources over fairness. It executes the shortest processes first to ensure that no process takes too long.